

Chapter 6. Crafting (Malicious) Input

As we have emphasized more than once thus far, the most interesting sorts of computational machinery are complex and therefore difficult to measure. Universal Turing machines, although themselves simple mechanisms of tapes, states, and readers, can compute incredibly intricate grammars. In theory, a Turing machine is capable of running any program that runs on today's most complicated computers. The problem is that understanding a real program in Turing machine terms (states, tape movements, and so forth) is not very useful. The level of explanation of a Turing machine exists at the wrong level and lacks in "big-picture" quality. Thus, the notion of what's really going on gets swamped by "irrelevant" detail. By analogy, consider trying to understand a game of billiards with reference to quantum physics. Although this should, in fact, be possible, a much better way of understanding billiards is to use Newtonian physics. Just as choosing an appropriate level of behavioral description is critical to billiards, it is critical to security as well.

Things get more complicated when we turn them "on." Chaos theory teaches us that simple dynamical systems (described in many cases by straightforward but iterative algorithms) give rise to complex behavior that is difficult to predict. Chaos theory gives us insight into modeling complex systems like the weather, but we're still not able to capture "open-ended" systems in a satisfying formal way. The problem is an explosion of possible future states, even in a system described by only a handful of equations. Because of this explosion of states, understanding and then securing an open dynamical system is extremely difficult. Programs running on modern networked computers are, in fact, open dynamical systems.

Generally speaking, software is driven by two basic factors: external input and internal state. Sometimes we can watch the external input to a program, perhaps by running a sniffer program or remembering what we type in the program's user interface. Much harder to discern is the internal state of a program, which encompasses all the bits and bytes stored in memory, registers, and so on. Behind the scenes, the software stores hundreds or thousands of pieces of information, some of which are data and some of which are instructions. This is kind of like a room full of thousands of tiny on/off switches. If we assume that it's possible to put every switch into every possible position in any combination, the sheer number of combinations gets huge fast (in fact, the number of combinations is exponential over the number of bits). There are so many combinations for a typical computer that it would take more particles than there are in the universe to store every possible state the computer can find itself in. The same goes for most modern software. Theory is not our friend, it seems.

The upshot of all this computer science theory is that most software is too complex to model. Treating software as a black box, we can type commands into the software for a lifetime and always know that, just around the corner, the next command we type might cause the software to fail. That's what makes software testing hard. Of course, in practice we know that certain strings of commands tend to cause software security failures. This is why there are a number of application security companies that sell software to run simple black box probes against an application, including Kavado, Cenzic, Sanctum, and SPI Dynamics. The thing is, because of the raging complexity of software, there is simply no way that any black box testing tool with baked-in tests can even begin to exercise every vulnerable state of a given program.

Software has lots of inputs. In a classic sense, traditional "input" takes the form of a sequence of commands or data bytes. This input drives software decision making by affecting state. The result of processing some input is usually some kind of output and a number of critical internal state changes. In all but the most trivial programs, this process is so complex that predicting software behavior over time becomes as hard as running the actual program itself.

Internal program state is analogous to the particular positioning of cogs and wheels inside a physical machine. A user of the machine can supply input—twiddled knobs and pushed

buttons in some sense—and drive the machine. The knob and button input becomes a language all its own—the programming language of the machine. Just like an Intel processor chip is a machine that executes x86 machine code, a software program is a machine that executes user input.

Clearly, the user can deeply affect the state of a running program by carefully crafting input—even malicious input meant to exploit the program. There is syntax and grammar to the input supplied by a user. There are certain commands that are rejected and others that cause deep state changes. There are potentially thousands of commands and millions of ways to combine these commands. Wielding the power of this language is the art of crafting input, and it is the subject of this chapter.

Think of an attacker as someone who wants the program to enter a certain vulnerable state. The attacker's main tool involves tweaking external input to the program. This input is, in some sense, a special variety of language that only the target program understands. So the target program is, in this line of thinking, a special machine designed to execute the attacker's statements. All of this leads us to the following conclusion:

A complex computational system is an engine for executing malicious computer programs delivered in the form of crafted input.

This conceptualization is very powerful. If you think about it, software programs in the wild are subject to exploit if just the right input arrives in just the right order. But, because of the complexity of the situation, it is very likely not possible to examine the software externally and determine whether such an exploit is possible.

The Defender's Dilemma

The external language defined by a computer program's input space is almost always more complex than the programmer imagines. One problem is that a program will interpret a command based on internal state that is extremely difficult to comprehend fully. To map the entire crafted input language at all possible internal states requires mapping all possible internal states as well as all possible logical decisions that affect state. Because the range of states is so large, the prediction becomes as difficult as running the program itself.

Attackers want to get the target program into a state in which crafted input causes a program to crash, allow code insertion, or run privileged commands. It's easy to find situations where this is possible. It's much harder to prove that none of these situations exist. Complexity is squarely on the side of the attacker, almost always ensuring success. How can you secure something against the unknown? Those who defend systems are in a horrible quandary: To defend a system properly you must think of *all* the attacks that could possibly be waged against you; but to attack, you need only find *one* unanticipated attack.

We know from logic that it is sufficient to disprove a proposition (e.g., that a system is secure) by demonstrating only one example in which the proposition is false (e.g., a successful penetration). On the other hand, it is not sufficient to prove the proposition by offering one or more specific examples in which the proposition appears to hold (e.g., unsuccessful penetration attempt).^[1]

[1] Proof by induction aside, of course.

Obviously, the job of the defense is extremely complex and may even be intractable in some cases. Underneath the apparent logic of a computational system lies this dragon of complexity. For years, some vendors of security products have conveniently ignored the true quandary, tending to overpromise and underdeliver based on a few simple cases.

Firewalls, virus checkers, and most IDSs are *reactive technologies, attempting to stop "dangerous" input from causing a vulnerable computation*. A better approach is to build a robust computation that does not require such shields. The nature of the problem is exacerbated by the difficulty of knowing what to block and what not to block. The problem is that there is no ultimate list of bad input to block because each program is unique in its "language."

You have heard this before, but it bears repeating: White listing, or exhaustively listing/defining all acceptable inputs, is a superior approach to black listing. Instead of trying to define all the possible bad things and stopping them, it is much better to define a list of allowed things and stick to it diligently. This is a version of the *principle of least privilege*. Give your program only as much power as it needs and no more. Don't give it too much power and later try to control it by blocking input.

Filters

Some software engineers who have only recently become security aware will attempt to add filters or special code to block "bad" requests.^[2] Instead of removing the very capability of the program to open privileged files in the first place, the programmer adds filters so the program will not accept "dangerous" filenames. Of course this approach is fundamentally flawed. How do you detect something that is "bad" if you don't know what "bad" really looks like? Can you create a universal rule to detect bad?

[2] This is a special case of a mechanism known as a *reference monitor*.

Consider this example. If user-supplied input is being delivered to a file system call, the engineer may block requests that have the string `../..` in them. The engineer is trying to

stop malicious use of the system call through a redirection attack. This simple attack is sometimes called *relative path injection*. An overly powerful file system call allows the attacker to download or access any file on the computer relative to the current directory. Typically the programmer will "fix" this bug by detecting when `../` occurs in an input string. But notice that this is just like intrusion detection, trying to detect the "bad." Depending on the rule the programmer hacks in, what happens when the attacker injects `.....//.....` instead, or codes the slash in hexadecimal unicode?

Communicating Systems

Think of all software as a system. Most targets are subsystems of a larger system. The target subsystem contains certain data that may be of value to the attacker. For example, the attacker may craft input that will cause a disclosure event from a subsystem.

Each subsystem also exists in relationship with other subsystems. The data contained within surrounding subsystems may be required to carry out a computation, but this allows the attacker possibly to subvert one weak subsystem to communicate with (possibly stronger) others. When thinking about subversion in this way, one must always consider the communication between systems as another layer of crafted input. The exact format and order of information being passed over subsystem boundaries is a dialect of crafted input language.

Intrusion Detection (Not)

One particularly clever way to craft input is to change the way a request looks as it traverses over the network. This can be accomplished easily by adding extra characters or replacing certain characters with other characters (or representations of characters). This simple kind of input crafting takes place all the time. Attackers who want to evade a simplistic IDS (and most of them these days remain simplistic) obfuscate an attack by using alternative character encoding and other related techniques. IDS evasion provides a classic example of using crafted input to your favor. Of course, crafted input can be used in many other ways to evade filters and/or cause logic errors.

Signature-Based versus Anomaly-Based IDSs

At their heart, IDSs are supposed to be conceptually similar to burglar alarms. The burglar breaks in, the alarm sounds, the authorities show up. This is reactive security at its apex. Businesses like Counterpane (a managed security service) exist to monitor IDS frameworks and deal with attacks.

There are two basic philosophies commonly found in IDS technology today—*signature-based approaches* and *anomaly-based approaches*. On one hand, signature-based technology relies on a database of known attack specifics. The idea is to compare traffic or logs, or some other input, with a list of bad things and to flag problems. So, in essence, signature-based technology detects *known* bad things. On the other hand, anomaly-based technology relies on learning what normal system behavior looks like and then detecting anything that doesn't fit the model. Anomaly-based technology detects "*not good*" things, where "good" is defined by the model. The approaches are fundamentally different.

A signature-based IDS must know explicitly about an exploit or an attack before it can be detected. Because of this, signature-based systems are easy to avoid, and savvy attackers skate by the IDS, do a little twirl in the air, and keep on going. If you know what features are used to set off alarms, you can avoid them. One thing that makes avoiding these systems particularly easy is the fact that most signature-based IDSs must know *precisely* what an attack looks like or else they simply don't detect anything. That's why simple tweaks to the input stream work so well for IDS avoidance.

An anomaly-based IDS doesn't really care what a specific attack looks like. Instead, it learns what normal patterns look like and then proceeds to find nonnormal patterns (anomalies). Anything that doesn't look normal enough gets flagged. The problem is (of course) that normal users don't always act and look the same. Thus, in practice, anomaly-based systems have a hard time separating novel but good from novel but not good. Clever attacks against anomaly-based systems using statistical windows are possible. One technique is to move the statistical profile from "completely normal" behavior very slowly into "attack space" in such a way that the model chugs merrily along, marking all behavior (including, ultimately, the attack) as normal.^[3]

^[3] This clever attack was first described by Teresa Lunt in a paper about the early intrusion detection system called *NIDES*. For more information, go to <http://www.sdl.sri.com/programs/intrusion/history.html>.

In the final analysis, a signature-based system can't catch anyone who is using the latest (presumably novel) attacks, and an anomaly-based system falls prey to the "cry wolf" phenomenon, and keeps catching normal users who are just trying to get their work done. Because impeding real work tends to get people fired and security systems thrown away, anomaly-based systems are almost never used in practice. And because people tend to forget about things they cannot see, feel, or taste, signature-based IDSs are fairly widely adopted despite their shortcomings.

Of course all IDSs can be used to create a diversion. One very common attack technique is to cause an IDS to "go ballistic" in one area of the network, while actually carrying out a clever attack elsewhere. Another common technique is to force an IDS to fire so often and with such regularity that it is eventually turned off in frustration. Then the real attack begins. Suffice it to say, many IDSs are not worth the money they cost, especially if the operating costs are factored in.^[4]

^[4] This point of view has been repeated by the Gartner analyst group in an often-cited report. Go to <http://www.csoonline.com/analyst/report1660.html> for an overview.

IDSs as a Reactive Subscription Service

Recall that almost all remote exploits against software rely on some sort of malformed transaction over the network. An attack transaction is usually unique in some way. IDSs bank on this concept. In fact, this is precisely what allows network IDSs to work at all. In practice, a network IDS is usually a network sniffer (think Snort) with a large set of trigger filters representing known attacks. The technology used in modern systems is, for the most part, no different than sniffer technology from 20 years ago. When put into action, trigger filters match various network packets that are thought to be malicious. These trigger filters are called *attack signatures*.

Obviously what we're talking about is a knowledge-driven model, which means that an investment in IDS equipment is only as good as the knowledge driving the system. This is a critical weakness. Without prior knowledge of the ins and outs of an attack, an IDS cannot detect the attack.

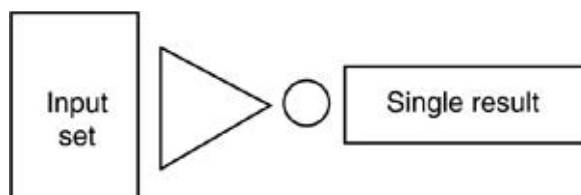
The main problem is that new exploits are discovered every day. This means that a network IDS is way too *reactive* to be effective. To keep up, the IDS must be constantly updated with a fresh signature database. Many IDS vendors supply a subscription service to update their customers with new signatures. The means, of course, that users implicitly trust an IDS vendor to provide meaningful and up-to-date attack data. In practice, this also tends to mean that users trust their IDS vendor to hire malicious hackers who sit in Internet relay chat (IRC) rooms all day trading "nfo" on the latest "0day spoils."

This is an interesting (and twisted) symbiotic relationship to be sure. Users of burglar alarms indirectly hire the burglars to update the very burglar alarms meant to catch the burglars they just hired. The reasoning seems to be that it's OK because the erstwhile good guys lurk under gray hats that obscure their faces.

The unfortunate truth is that no IDS will ever know about real 0day exploits. Generally speaking, IDS vendors will never find out about the latest vulnerabilities. Some past vulnerabilities were known about literally for years in the hacker underground before they were ever reported publicly. Take BIND as an example: Certain groups in the hacker underground had full knowledge of various buffer overflows in BIND for *several years* before the problems were finally revealed publicly and then patched.

The Effect of Alternate Encoding on IDSs

There are hundreds of possible ways to encode a single attack, and each looks different on the network, even though each produces *exactly the same result*. *This is convergence of input onto a given state*. There exists a large and varied set of input that drives a target program into a single result state. In other words, there is not a clear one-to-one relationship between a given input value and a given state (for most programs). There are, for example, millions of different packets that can be injected into a system where the system ends up ignoring the input. More to the point, there are usually thousands of packets that always result in the same real response from a target program.



To function properly, a network IDS must know every encoding and every permutation of input that will result in a successful attack (for every given attack signature). This quickly becomes intractable. As an upshot, by using only simple rules, an attacker can twist standard-issue attacks into so many knots with so many layers that by the time the IDS chainsaws its way through the mess, the attacker is sipping tequila in Bermuda.

In [Figure 6-1](#) we illustrate a type of desynchronization that was used with great effect in the late 1990s. The GET request is segmented over several packets. Both requests—labeled A and B—are sent to the target. At the bottom of these requests is the packet number in which the data arrives. In both requests, ten total packets are sent. However, we can see that the characters sent are slightly different. Request A is mangled while request B is a legitimate GET request for the cgi-bin directory.

Figure 6-1. Desynchronization along packet boundaries with a GET request.

A:	G	T	E		/	c	X	i	-	b	i	n
	1	2	3	4	5	6	7	8	9	10		

B:	G	E	T		/	c	g	i	-	b	i	n
	1	2	3	4	5	6	7	8	9	10		

C:	G	T	T		/	c	X	i	-	b	i	n
----	---	---	---	--	---	---	---	---	---	---	---	---

D:	G	E	T		/	c	g	i	-	b	i	n
----	---	---	---	--	---	---	---	---	---	---	---	---

E:	G	T	E		/	c	g	i	-	b	i	n
----	---	---	---	--	---	---	---	---	---	---	---	---

Compare requests A and B. Notice that there are overlapping packets. For example, packet 1 includes both "GT" and "G." Packet 2 includes both "ET" and "E." When these packets arrive at the target, the target must figure out how to resolve the overlapping characters. There are several combinations that are possible. The strings labeled in the diagram as C, D, and E are all valid reconstructions of the final string. The attack against an IDS occurs when the IDS reconstructs a mangled or unintelligible string, while the server target reconstructs a valid request.

This problem is made exponentially worse for each protocol layer that allows overlaps to occur. Using fragmentation, the IP protocol layer can be overlapped in this way. Using segmentation, the TCP protocol can be overlapped in this way. Some application-layer protocols allow even further overlap. If an attacker combines several layers of overlap in multiple layers of protocol, the possibilities for reconstruction are large (to say the least).

Any IDS that hopes to try every possibility of a request is clearly at a loss. Some IDSs attempt to model the behavior of each target and thus provide more accurate reconstruction. This assumes the model of the target is accurate, which is a difficult problem in its own right. And this also assumes that even with a working model for the target, the IDS can provide the reconstruction at a supposed gigabit line speed. In practice, IDSs simply mark such an obfuscated request as "interesting," but almost never reconstruct anything of value about its content. At the heart of this issue is a question of protocol clarity. Application-layer packet construction is the difficult issue to solve. At the bottom of the food chain, TCP/IP is very clearly defined, so an IDS can generally reassemble packet fragments at very high speed (often in hardware). Well-coded IDSs can sometimes do a decent job with simple protocols like HTTP as well. But application-specific reconstruction is very difficult and remains beyond the grasp of most IDSs.

Partition Analysis

A complex software system can be viewed as a collection of subsystems. One could even view the Internet as a single (albeit exceptionally large) software system. Each computer attached to the Internet could, in this view, be understood as a subsystem. These computers can, of course, be further subdivided into subsystems themselves. The process of taking a large system and slicing it into smaller, easily digestible chunks is called *partitioning*. A typical system can be partitioned at many different scales.

Clearly we cannot address a system with infinite bounds, so we are always dealing with software that exists inside a large, definable whole. This is fitting because the entire universe is a (bounded) collection of systems that pass information.^[5] In theory, there is no actual end to an exploitable application that is targeted. One great technique is to create artificial partitions with which to measure success. The easiest place to start is the executing process—the image of the software as it looks on a particular machine at runtime. Using tools described in this book, you can measure a software process and determine its loaded code modules. Likewise, you can sniff input and other traffic to discern communications between modules, the OS, and the network. You can also see output communications with the file system, external databases, and outgoing connections on the network. Lots of data to ponder.

^[5] We assume the closed model of the universe with a start at the Big Bang.

Even this process itself can be subdivided into partitions. For example, we can treat each DLL as a separate unit, and analyze each one separately. Then we can analyze the input and output of a smaller partition by hooking various API calls.

The following example illustrates API hooking on the Windows platform. Note that in [Chapter 3](#) we discuss how you can write your own *call hook tools* from scratch.

APISPY for Windows Revisited

Almost all platforms provide or otherwise have associated tools to trace API calls. We discuss Truss under Solaris in [Chapter 4](#), for example. The Wintel platform also has many tools. Recall from [Chapter 3](#) our use of APISPY32 to uncover all calls to `strcpy` made by the Microsoft SQL server target program. Recall that we chose this call because if the source string can be controlled by an attacker, a buffer overflow exploit may be possible. Our simple example involves simultaneous sampling between two "partitions": the SQL executable and a system DLL called `KERNEL32.DLL`.

A straightforward way to start reversing software is to take inventory of all input and output points, looking for an interesting partition. At the time of this writing, there are few good tools that help you manage the kind of auditing process we're looking for.^[6] You might make a spreadsheet or write a tool to keep track of all calls that take user input. Most attackers use pencil and paper to scratch down addresses that call interesting functions like `WSARecv()` or `fread()`. A tool like IDA-Pro will let you comment the dead listing, which is certainly better than nothing. When examining the code, be sure to take note of all output points as well, including calls to functions like `WSASend()` and `fwrite()`. Note that output sometimes takes the form of system calls.

^[6] Program analysis tools such as code surfer and other tools that allow data flow and control flow analysis are a big start.

Red Pointing

The easiest and quickest method for reverse engineering code is known as *red pointing*. An

experienced reverse engineer simply reads through the code looking for obvious weak points, such as calls to `strcpy()` and the like. Once these target areas have been identified, these locations are sampled and a concerted attempt is made to get the program to hit the location during execution. Using an API call-hooking tool and dynamic execution is the easiest way to do this. If the specific code locations in question cannot be easily hooked using a simple tool, then a debugger can be used.

Two things combine to make a red point in the target code: a weak location with a potentially vulnerable system call, and user-supplied data that flow to and are processed at the location. Without performing an explicit, detailed input trace, the dynamic process outlined here is part luck. A little experience helps in finding possible weak locations and deciding what input might get processed at a given target location. This gets easier with practice.

The most notable feature of red pointing is its general ease. However, the "ease" of this approach may be less appealing after red pointing for a few hours and not getting any hits. Sometimes red pointing can be discouraging. On the other hand, sometimes you can find vulnerable code almost immediately using this technique. Your mileage will definitely vary.

The big downside to red pointing is that it tends to miss all but the most trivial of bugs. Then again, lots of software appears to have these bugs, making even this simple technique very effective.

To improve your odds with this approach, we introduce several techniques in the following pages. All these techniques can be combined of course. These techniques start with red pointing and move deeper into the code using backtracing, leapfrogging, and input tracing.

Tracing Code

No matter how much software exploiters would like everything to be as easy as red pointing, the fact is that if you want to find interesting exploits, you'll probably need to get your hands dirty in the code itself. This means tracing input—a dirty job and a very tiresome one to boot. One reason why many simple vulnerabilities remain in fielded software is that nobody has the patience to review the software fully in quite the same way that an attacker does. Even automated tools are not yet good enough to find all the vulnerabilities.

The human mind is dreadfully slow, but remains the best pattern-matching system we know. Most vulnerabilities are not completely schematic and algorithmic—that is, they do not tend to follow an easy-to-recognize pattern that can be codified into a tool. This means that automated scanners cannot find them. *Human auditors are still the best tool to find exploits.*

The problem is that humans are not only slow, they are very expensive to operate. This means that finding exploits is still a relatively expensive affair. Regardless, such auditing is usually worth the expense. A vulnerability in the field can easily cost a software vendor more than \$100,000 to deal with, especially considering public relations, patch deployment, and technical support—not to mention the danger of providing keys to the computer kingdom for some attacker. On the other side of the coin, as an attacker, having exclusive access to a remote root exploit is in fact like having the keys to the kingdom (especially if the 'sploit in question applies to a widely used program like BIND, Apache, or IIS).

Backtracing from Vulnerable Locations

Let's assume we have determined some meaningful partitions and begin analyzing them for weaknesses. Using our call-hooking trick is easy: Simply run the code on some test input and hope that you see the data being used in the suspect call. Of course, things aren't this easy in the real world. In the most usual scenario, you will need to craft your input using special characters and/or certain types of requests.

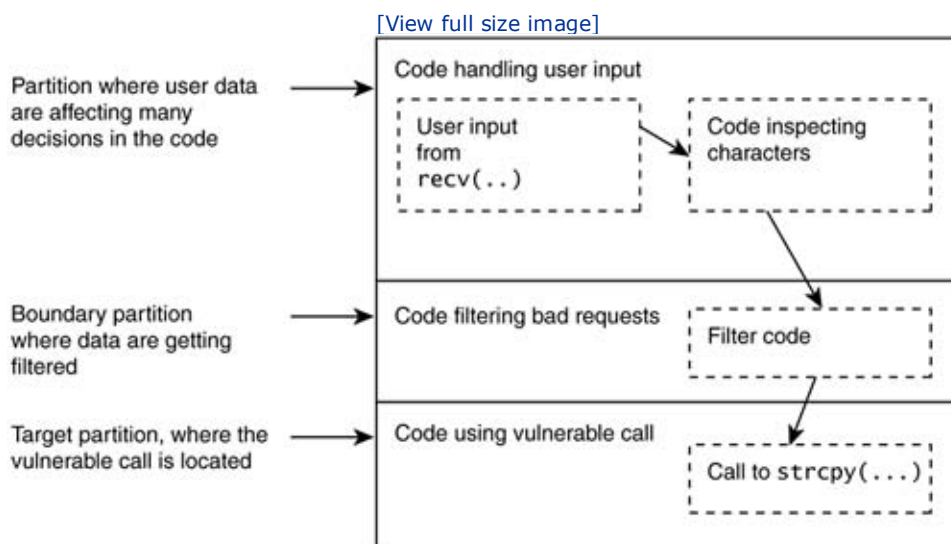
In any case, the current goal is to find weaknesses that can be exercised from outside the partition—that is, via input passed over the partition boundary. For example, if we partition at DLL boundaries, then we will want to find all vulnerabilities that can be exercised via the exported function calls into the DLL. This will be particularly useful because we can then look at all programs that use the DLL and determine how any vulnerability we uncover affects them.

The first step to backtracing is to identify potentially vulnerable calls. If you aren't sure if a given call is vulnerable, write a small program to test the call itself. This is a great way to learn. Then write a separate program that supplies all possible inputs as the arguments with the results sent to an output call. Figure out which arguments cause trouble and go from there. Perhaps your toy program will crash, or the output call may do something that would be considered a security violation (say, reading a file). You will want to map the characters that cause problems for the call (which we call the *hostile character set*) and any strings that cause problems for the call (which we call the *hostile statement set*). Once you determine the hostile character and statement sets, you can begin backtracing in the target program to determine whether either set can be applied by an attacker from outside.

To begin moving backward from your target location, instrument the target program at points further up the code's control flow tree (usually by setting break points with a debugger). Then inject input using the hostile character and statement combinations (with a client program). If the inputs you try can reach the call, then you're in business. You can consider this a newly expanded "vulnerable partition." Notice that we're growing things outward from the internal vulnerability. As soon as an injection point at a new boundary location results in malicious input being blocked, by our definition you will have traversed partitions.

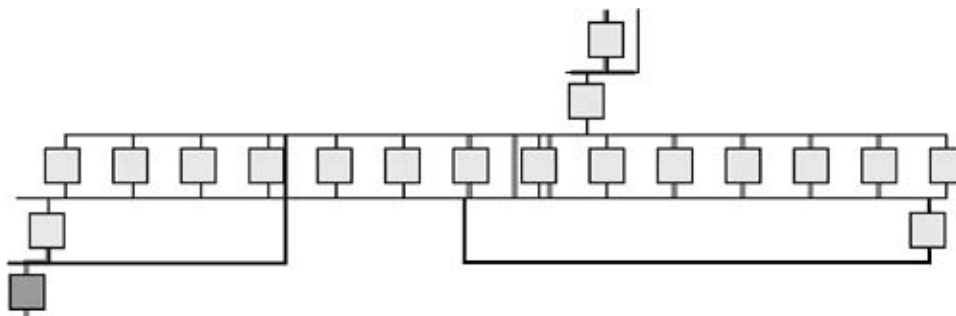
[Figure 6-2](#) illustrates three partitions. The first handles user input, which is then filtered and possibly blocked in the second, before we can reach our goal—the third partition (which includes the vulnerable location). Harking back to our previous example, we want the DLL boundary to be hit *before* we traverse out of the vulnerable partition.

Figure 6-2. Three partitions in a target and their effect on backtracing.



[Figure 6-3](#) shows a code backtrace in the IRC.DLL supplied with Trillian—a popular chat client. The vulnerable location we're zeroing in on contains a sign mismatch error. The backtrace shows a large switch statement that occurs above the suspect location.

Figure 6-3. The dark gray box in this picture represents a vulnerable location in the Trillian IRC.DLL code. Control flows through a large switch statement on its way to the location in question. We used IDA-Pro to make this picture.



The goal is to connect user input to the vulnerable location. One valid approach is keep backtracing until you hit a known input point, such as a call to `WSARecv`. If you can trace back to a call like this while remaining in the "vulnerable partition" using hostile statements, you have uncovered a real, live vulnerability. (Note that the type of analysis we're describing is

tedious and time-consuming.)

If you find the process of backtracing too laborious, another method is to backtrace until you can clearly identify a set of coarse partitions. You can then trace forward from real input points to determine whether any of the coarse partitions you have defined can be reached. In this way you can extrapolate your way to possible attacks by working both sides. If a vulnerable partition can be reached using hostile statements, then it follows that the hostile statement may be able to make it completely from an initial input location to the final desired output event.

All such hypotheses must be directly tested, of course, but identifying possible attacks as we have described certainly helps. This approach is much less haphazard than simply splatting the inputs of a program with "possible attacks" in a simple-minded black box fashion (which is precisely what many of the early application security tools on the market do today).

Dead Ends and Runouts

One huge dissatisfying problem with static backtraces is that they have a tendency to run out. That is, you're chugging merrily along in an analysis and suddenly you hit a dead end. Perhaps you can't figure out where data arrive from. One way around this kind of local snag is to run the program and observe the code at the dead end directly.

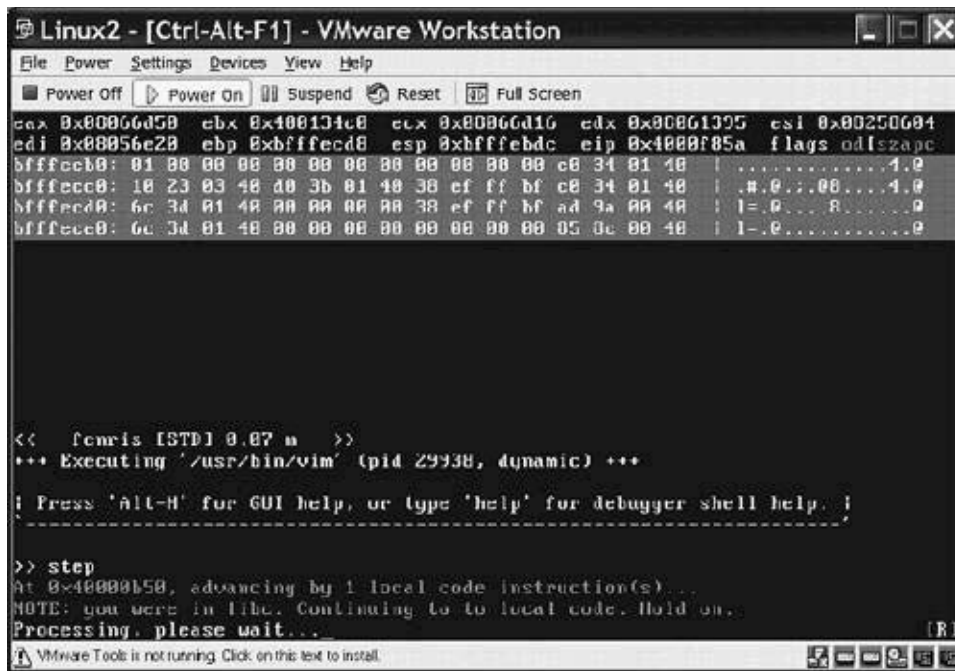
One example where this can be useful is in Windows message pumps. If you backtrace and hit a Windows message handler, determining where the messages originate (are posted from) can be difficult. Fortunately, at runtime you can usually see exactly where the message is posted from, because the data you need will be found in the call stack.

Runtime Tracing

Runtime tracing involves setting break points and single-stepping code during runtime to build a model of the program. At runtime you can trace data flow and control flow in a naive fashion simply by watching what happens. For complex code this is usually much more practical than any kind of pure static analysis. At the time of this writing, there aren't many tools available that assist in runtime tracing, especially for security problems. One tool that shows much promise is called *Fenris*, and is available for the Linux platform ([Figure 6-4](#)).

Figure 6-4. A screen shot of Fenris running in a VM. Fenris is a useful runtime tracing tool.

[\[View full size image\]](#)

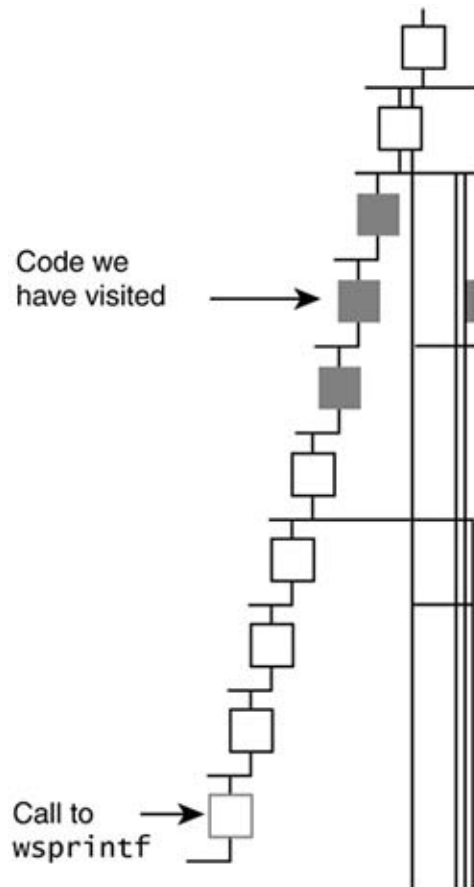


The notion of code coverage is central to runtime tracing. The idea is that you want to visit all the possible places where things can go wrong (that is, you want to cover them).^[7] In many (often frustrating) cases you will find a potential vulnerability, but you won't be able to reach it. If this happens you will want to keep modifying possible hostile input until you reach the location in question. The best way to do this is to wield a code coverage tool.

^[7] In testing terminology, the coverage criteria we're after here is potential vulnerability coverage.

In [Figure 6-5](#), the location we want to hit contains a call to `wsprintf`.

Figure 6-5. Results from our simple vulnerability coverage tool. Covered code segments are in gray. We have not yet found a path to the vulnerable box (which includes a call to `wsprintf()`).



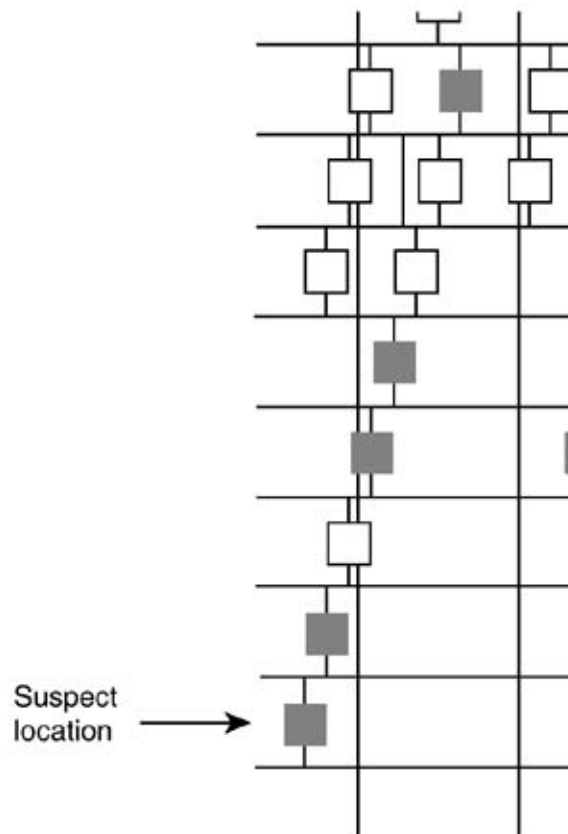
Code locations that we have successfully visited so far are shown as gray boxes.

To measure coverage over particular code locations, we have constructed a simple tool that combines IDA-Pro and a debugger. Specific locations are obtained from IDA-Pro using a custom plugin. The locations are then measured at runtime by setting break points at the beginning of each code location in the debugger. When a break point is hit, the location is highlighted in gray.^[8]

^[8] Source code for the coverage tool mentioned here can be obtained from <http://www.hbgary.com>.

By tuning input and watching how certain branching decisions are being made, an attacker should be able to craft input so that it reaches the potentially vulnerable location. Instantaneously zeroing in on a vulnerable location (as shown in [Figure 6-6](#)) almost never happens quickly. The attacker must very carefully analyze how each branching decision is being made in the code and manipulate input accordingly. This requires lots of time in the debugger.

Figure 6-6. Here, the suspect location has in fact been hit using some crafted input. Success!



Speedbreaks

In many cases, directly sampling data in memory can help to determine when a certain code location is hit. This is a convenient technique. Sometimes we can set things up to do this automatically whenever a break point is hit. We call this a *speedbreak*. When the break point we're interested in is hit, each register is examined. If the register points to valid memory address, then a sample of the memory is taken. This technique tends to reveal how parsers are using strings and how character conversions are taking place. It can even be used to trace user-supplied input.

On a Windows machine, the technique is fairly simple: Each register value is supplied in the context structure when a debug event occurs (see [Chapter 3](#)). For each register, the debugger calls `VirtualQuery()` to determine whether the memory address is valid. If so, a sample is taken, and the program is allowed to continue execution.

[Figure 6-7](#) shows a simple speedbreak tool being used to sample an FTP server. We see a SQL query being constructed in memory. This tool is available to the public domain and is registered at <http://www.sourceforge.net> (see [projects/speedbreak/](#)).

Figure 6-7. A simple speedbreak tool used to sample an FTP server's memory use. The column on the left indicates the time at which the sample was taken.

[\[View full size image\]](#)

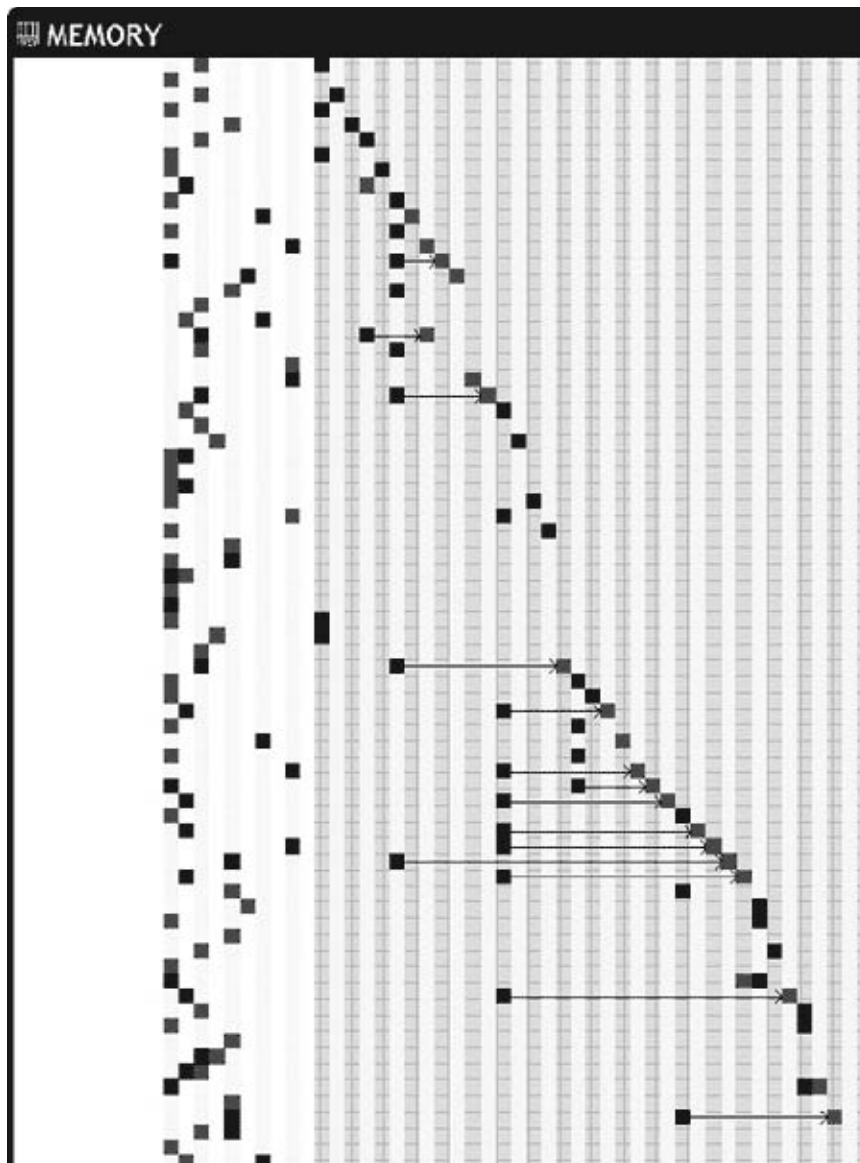
Hits		EAX: 08984058(144195672)	-> SELECT * FROM ACCOUN
Time: 12:25:57:257		EBX: 00B4F0F4(11858164)	-> .w.I.L.
Time: 12:25:57:257		ECX: 00000014(20)	
Time: 12:25:57:257		EDX: 00000014(20)	
Time: 12:25:57:257		ESI: 00B4F7AC(11859884)	-> X@.I.k>I...
Time: 12:25:57:257		EDI: 0000002A(42)	
Time: 12:25:57:257		EBP: 004A0604(4851204)	-> SELECT * FROM GROUPS
Time: 12:25:57:257		ESP: 00B4F0C0(11858112)	-> X@.IIIJ
Time: 12:25:57:257		+0: 08984058(144195672)	-> SELECT * FROM ACCOUN
Time: 12:25:57:257		+4: 004A0604(4851204)	-> SELECT * FROM GROUPS
Time: 12:25:57:257		+8: 00B4F0F4(11858164)	-> .w.I.L.
Time: 12:25:57:257		+12: 77121644(1997674052)	-> .D\$if.
Time: 12:25:57:257		+16: 003E4F50(4083536)	-> .SJ

Tracing a Buffer

One reasonable method for tracing input is to set a break point in the code at the location where the input buffer is located. From this point, you can single step the code forward and trace wherever the input buffer in question is accessed or copied. The Fenris tool supports this kind of tracing. In our tool kit, we have a simple tool that performs this kind of tracing under Windows.

[Figure 6-8](#) shows a memory trace. Using this visualization technique we can track a single buffer of input over time. The basic idea is to determine when and where data move from registers to stack and heap locations with reads and writes. Knowing where our data end up is a great help in crafting an exploit.

Figure 6-8. A memory trace shows registers (on the left) and stack and heap memory (on the right). Darker squares indicate the source of a read operation. Lighter squares indicate the target of a write operation. Arrows indicate source and destination in a move operation. This tool was internally developed by Høglund, and at the time of this writing has not yet been released. Check <http://www.sourceforge.net> for updates.



Leapfrogging

Leapfrogging is a shortcut for input tracing. Instead of tediously tracing through every line of code, you set memory-read break points on the user-supplied buffer. The Intel x86 family of processors supports debugging break points for memory access. Unfortunately not all standard debugging programs expose this functionality. Two good tools that can be used for setting memory break points are SoftIce and OllyDbg.

As is the case with input tracing, a break point is set on the input point in the program. When the buffer is read from the user, a memory-read break point can be placed on that buffer. You then allow the program to continue running. At this point, we have no idea which code paths are being exercised or how control flow works in the target. What matters is that if *any* of the code attempts to access the user buffer, the program will halt and you can determine the line of code that is attempting the access. Although this technique is not as effective as tracing code manually (because much less understanding about program behavior is gleaned), we still have the benefit of noting every location that reads data from the user buffer.

The leapfrogging method is not foolproof. The fact is that data are copied from the user

buffer all the time. Whenever this occurs, we'll get a break point, but the data that are copied end up populating other memory locations and CPU registers. Unless you single step, you cannot see what happens to the data after they leave the user buffer. To perform a complete analysis requires setting additional memory break points *on all the snippets of data that get copied*. Needless to say, that's lots of break points. Because the Intel processor only supports four memory break points, you will quickly run out of trapping options. In a complex program, the data propagation quickly becomes intractable for a manual approach like the one we have described. However, using a combination of leapfrogging and input tracing provides plenty of data to the reverse engineer.

The upside to leapfrogging is that some exploits can be found this way. The downside to this approach is that the technique is very likely to miss complex problems. Interestingly, this means that the leapfrogging technique is much more useful for attackers than it is for defenders.

Memory Page Break Points

A variation of the leapfrog involves changing the protection on large swaths of memory. Rather than use a particular memory break point, the debugger changes the memory protection on the whole page in memory. If code tries to access the marked page, an exception will occur. The debugger can then be used to examine the event and determine whether the user-supplied buffer is being moved around. OllyDbg supports this kind of course-grained break point.

Boron Tagging

Another excellent time-saving technique is called *boron tagging*. With this technique, either in response to a single-step event or in response to a memory-read break point during a leapfrog, the debugger is set up to examine the memory pointed to by all the registers. If a predefined substring exists in any of the samples, then the location is subsequently marked as handling "user-supplied input" (an interesting location). The trick is, of course, to supply the particular magic substring in your attack input (hoping that it successfully propagates through the program to your detection point). If you're lucky, you will get a map of all the locations that handle user input. Of course, if the substring is ignored or converted to something else before it gets anywhere interesting, this technique will not work.

Reversing Parser Code

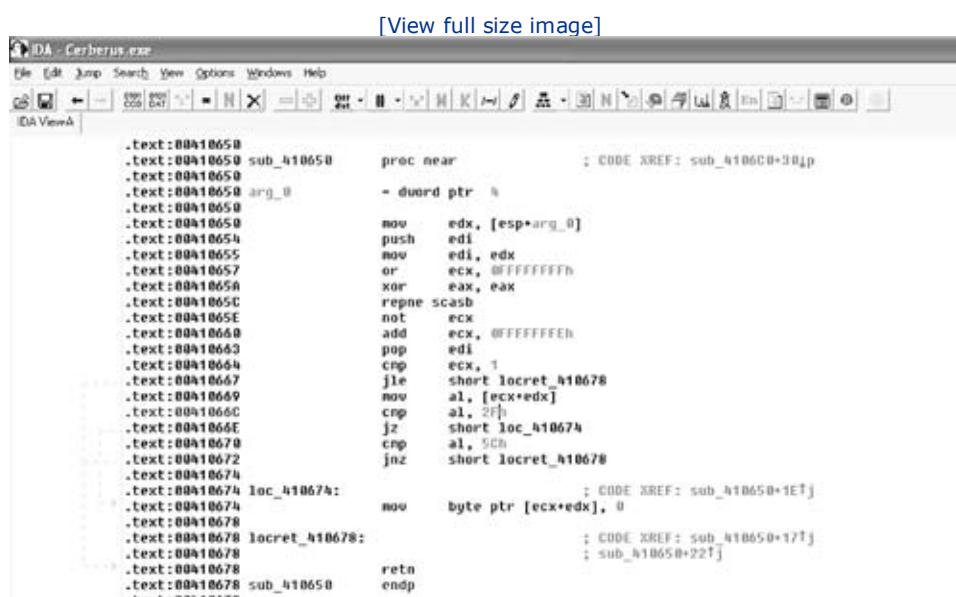
A parser breaks apart a raw string of bytes into individual words and statements. This activity is called *parsing*. Standard parsing usually requires "punctuation" characters, often called *meta-characters* because they have special meaning. Many times, target software will parse through an input string looking for these special characters.

Meta-characters are often points of interest for an attacker. Many times important decisions rely directly on the presence of these special characters. Filters also tend to rely on meta-characters for proper operation.

Meta-characters are often quite easy to spot in a dead listing. Spotting them can be as simple as looking for code that compares a byte value against a hard-coded character. Use an ASCII chart to determine the hex values for a given character.

In the IDA screen shot shown in [Figure 6-9](#), we can see two locations where data are being compared with the forward slash and back slash characters—2F and 5C, which map to / and \ respectively. These kinds of comparisons tend to crop up in file system filters, and thus make interesting starting places for an attack.

Figure 6-9. An IDA disassembly of a common FTP server showing the comparison for slash characters 2F and 5C.

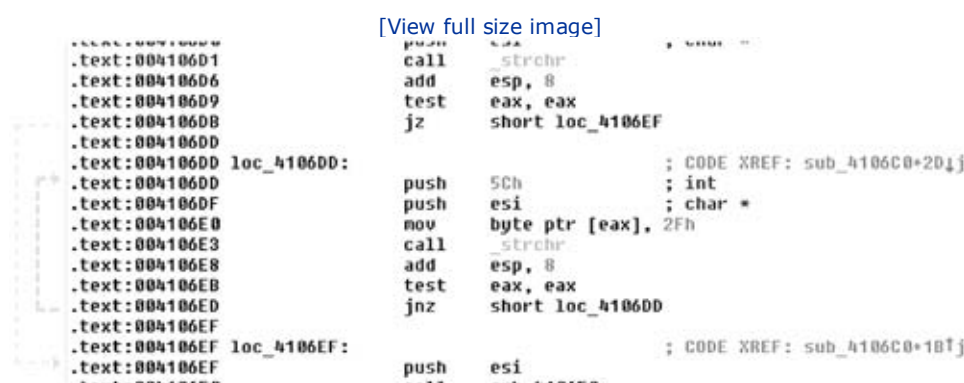


Character Conversion

Character conversions sometimes occur as a system prepares itself to make an API call. For example, although a system call may expect a file system path to be supplied using forward slashes, the program may accept both back slashes and forward slashes to mean the "same thing." So, the software converts back slashes to forward slashes before making the call. This kind of transformation results in equivalent characters. It doesn't matter which kinds of slashes you supply, they will be treated as forward slashes to the system call.

Why is this important? Consider what happens if the programmer wants to make sure the user can't supply slashes in a filename. This might be the case when the programmer is trying to prevent a relative path traversal bug, for example. The programmer may filter out forward slashes and believe that the problem is solved. But if an attacker can insert a back slash, then the problem may not have been properly handled. In situations in which characters are converted, an excellent opportunity exists to evade simple filters and IDSs. [Figure 6-10](#) shows code that converts back slashes to forward slashes.

Figure 6-10. The code here is using an API call `strchr` to find character `5Ch (\)` in a string. Once the character is found, the code uses `mov byte ptr [eax], 2Fh` to replace the back slash with character `2Fh (/)`. This loops until no more back slashes are found (via the `test eax, eax` and subsequent `jnz`, which jumps [if not zero] back to the beginning of the loop).

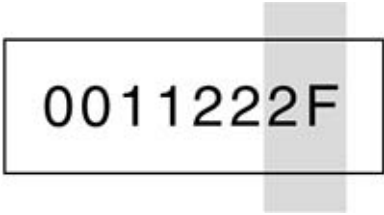


Byte Operations

Parsers built into most programs usually deal with single characters. A single character is generally encoded as a single byte (the clear exception to this rule being multibyte/unicode characters). Because characters are usually represented as bytes, identifying single-byte operations in a reverse assembly is a reasonable undertaking. Single-byte operations are easy to spot because they use the notation "al," "bl," and so forth. Most registers today are 32 bits in size. This notation indicates that operations are being performed on the lowest 8 bits of the register—a single byte.

There is a classic "gotcha" here to keep in mind when debugging a running program. Remember that only a *single byte* is being used with notations like al and bl, regardless of what exists in the rest of the register. If the register has the value `0x0011222F` (as shown in [Figure 6-11](#)), and the byte notation is being used, the actual value processed is `0x2F`, the lowest 8 bits.

Figure 6-11. A single byte (2F) as represented in a 32-bit register.



0011222F

Pointer Operations

Strings are often too large to be stored in a register. Because of this, a register will usually contain the address of the string in memory. This is called a *pointer*. Note that pointers are addresses that can point to almost anything, not just strings. One nice trick is to find pointers that increment by a single byte, or operations that use a pointer to load a single byte.

Byte operations with pointers are easy to spot. Pointer operations follow the [XXX] notation (for example, [eax], [ebx], and so on) in combination with the al, bl, cl, and so forth, notation.

Pointer arithmetic has the notation

`[eax + 1], [ebx + 1], etc.`

Moving bytes around in memory ends up looking something like this:

```
mov dl, [eax+1]
```

In some cases, the register where the pointer is stored is modified directly, like this:

```
inc eax
```

NULL Terminators

Because strings are typically NULL terminated (especially when C is being used), looking for code that compares with a 0 byte can also be useful. Tests for the NULL character tend to look something like this:

```
test al, al
```

```
test cl, cl
```

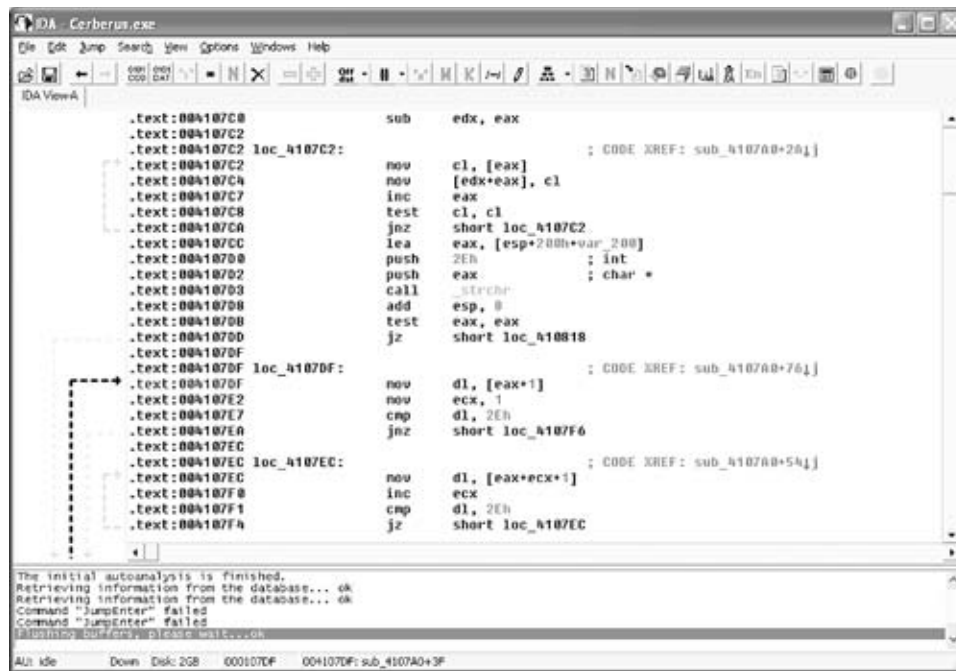
and so forth.

[Figure 6-12](#) includes several single-byte operations:

- `cl`, byte notation
- `[eax]`, a pointer
- `inc eax`, increment pointer
- `test cl,cl`, looking for NULL
- `[eax+1]`, pointer + 1 byte
- `mov dl,[eax+1]`, moving a single byte

Figure 6-12. Code with several interesting 1-byte operations included.

[\[View full size image\]](#)



These operations may indicate that the program is parsing or otherwise processing input.

Example: Reversing I-Planet Server 6.0 through the Fro Door

Like most server software, Sun Microsystems I-Planet 6.0 software uses a "detect the bad" black approach to security. As we have made clear, such an approach is easily defeated. Using call tracing and GDB (described in [Chapter 3](#)), we locate several function calls meant to filter user-supplied input. Instead of simply rejecting malicious input, the I-Planet server attempts to "correct" malicious input by removing the "bad" parts.

In this particular case, the most effective approach to find these functions involves break points and the "outside-in" approach. Remember from [Chapter 3](#) that going outside-in means beginning a trace when user input is accepted, and attempting to move forward into the program.

Working outside-in, we discover an often-used function called

```
__0fJCHttpUtilTCanonicalizeURIPathPCciRPcRiT
```

The name of the function is certainly mangled, but we can see that it's used to canonicalize (or convert to standard form) the user-supplied URI string. As we have mentioned, this function is designed to filter "bad" input strings. Using GDB to set a break point at the beginning of this function, we can examine the data that are being supplied:

```
(gdb) break __0fJCHttpUtilTCanonicalizeURIPathPCciRPcRiT
```

```
Breakpoint 6 at 0xff22073c
```

```
(gdb) cont
```

```
Continuing..
```

A break point is now set, but we still need to issue a request to determine which data arrive at the function. We issue a Web request to the target and the break point promptly fires. We examine the registers with the command `info reg` to determine which data are supplied:

Breakpoint 6, 0xff22073c in __0fJCHttpUtilTCanonicalizeURIPathPCciRPcRiT ()

from /usr/local/iplanet/servers/bin/https/lib/libns-httpd40.so

(gdb) info reg

g0	0x0	0
g1	0x747000	7630848
g2	0x22	34
g3	0x987ab0	9992880
g4	0x98da28	10017320
g5	0x985a18	9984536
g6	0x0	0
g7	0xf7641d78	-144433800
o0	0x985a8c	9984652
o1	0x15	21
o2	0xf7641bec	-144434196
o3	0xf7641ad4	-144434476
o4	0x0	0
o5	0x987ab0	9992880
sp	0xf7641a48	-144434616
o7	0xff21ae08	-14569976
l0	0x985390	9982864
l1	0xff2d80d0	-13795120
l2	0x987aa0	9992864
l3	0x336d38	3370296
l4	0x985a28	9984552
l5	0xff2d7b38	-13796552
l6	0x987aa0	9992864

```

17          0x987ab0 9992880
i0          0x985a88 9984648
i1          0x2000      8192
i2          0x9853ac 9982892
i3          0x987ab0 9992880
i4          0x985584 9983364
i5          0x1          1
fp          0xf7641bf0      -144434192
i7          0xff21938c      -14576756
y           0x0          0
psr         0xfe901001      -24113151      icc:N--C, pil:0, s:0, ps:0, et:0,
wim         0x0          0
tbr         0x0          0
pc          0xff22073c      -14547140
npc         0xff220740      -14547136
fpsr        0x420      1056      rd:N, tem:0, ns:0, ver:0, ftt:0, qne:0, fcc:<, i
                                cexc:0
cpsr        0x0          0

```

Next we examine each register with the `x` command. A convenient trick is to use the `"x/"` notation to dump the memory around the address in question. The command `x/8s $g3`, for example, dumps strings around the memory pointed to by register `g3`:

```

(gdb) x/8s $g3
0x987ab0:      "GET /knowdown.class%20%20 HTTP/1.1"
0x987ad3:      "unch.html"
0x987add:      " "
0x987ade:      " "

```

```
0x987adf:      ""  
0x987ae0:      ""  
0x987ae1:      ""  
0x987ae2:
```

Our supplied URI is stored in a memory location pointed to by the g3 register. We can now begin stepping forward and taking notes in IDA.

This outside-in approach is particularly well suited to finding parsing tricks. Usually input data is "frobbed" and otherwise modified by the time they reach an interesting system call. By starting outside, we can determine what the parser logic is doing to the data. For example, extra slashes be stripped from a filename. The request might not be forwarded if certain character sequences present (such as our redirection-invoking string `../..`).

[Figure 6-13](#) shows an IDA screen shot with notes appended to interesting locations. The output GDB can be directly pasted into the IDA disassembly. Pressing the semicolon key in IDA allows repeatable comments to be entered. By tracking the call, we find that many characters are stripped and that the filename is in this (broken) way "cleaned up."

Figure 6-13. An IDA screen with notes appended to the code. Keeping of work in IDA is essential.

[\[View full size image\]](#)

```

IDA - libns-httpd40.so
File Edit Jump Search View Options Windows Help
IDA View
text: 000A073C | Attributes: bp-based frame
text: 000A073C |
text: 000A073C | global __0fJCHttpUtilCanonicalizeURIPathPcRt
text: 000A073C | __0fJCHttpUtilCanonicalizeURIPathPcRt:
text: 000A073C | save %sp, -0x60, %sp | CODE XREF: __0fJCHttpUtilCanonicalizeURIPathPcRt
text: 000A073C | call __INTsystem_malloc | %sp has the string of the URI
text: 000A0740 | add %i1, 1, %p0
text: 000A0744 | mov 0, %i5
text: 000A0748 | orcc %p0, %p0, %p2
text: 000A0750 | mov %i0, %i4
text: 000A0754 | bne.pn %i0cc, loc_A0768
text: 000A0758 | mov %p0, %p3
text: 000A0760 | st %i5, [%i2]
text: 000A0764 | ba locret_A0910
text: 000A0764 | clr [%i3]
text: 000A0768 |
text: 000A0768 | loc_A0768: | CODE XREF: __0fJCHttpUtilCanonicalizeURIPathPcRt
text: 000A0768 | mov 1, %p5
text: 000A076C | mov 2, %p0
text: 000A0770 | mov 3, %p4
text: 000A0774 | cmp %i5, %i1
text: 000A0778 |
text: 000A0778 | loc_A0778: | CODE XREF: __0fJCHttpUtilCanonicalizeURIPathPcRt
text: 000A0778 | bge.pn %i0cc, loc_A0974 | __0fJCHttpUtilCanonicalizeURIPathPcRt+0011
text: 000A077C | sub %p3, %p2, %p2
text: 000A0780 | ldsb [%i4], %p1 | 14 increments forward one in this loop
text: 000A0780 |
text: 000A0780 | (gdb) x/8s $i4
text: 000A0780 | 0x8b4814: '/' (repeats 28 times), "core.c H
text: 000A0780 | 0x8b48dc: "age: en-us\r\nAccept-Encoding: g
text: 000A0780 | 0x8b4978: "1s/4.0 (compatible; MSIE 6.0; W
text: 000A0780 |
text: 000A0780 | from:
text: 000A0780 | http://dat.lab.local/../../../../co
text: 000A0780 |
text: 000A0780 | compare to '/'
text: 000A0784 | cmp %p1, 0x2F
text: 000A0788 | be.pn %i0cc, loc_A07E4
text: 000A078C | mov %p3, %p2
text: 000A0790 | inc %i5
text: 000A0794 | inc %i4
text: 000A0798 | stb %p1, [%p2]
text: 000A079C | inc %p3
text: 000A07A0 | inc %p4
text: 000A07A4 | inc %p0
text: 000A07A8 | inc %p5
text: 000A07AC | ba loc_A0778 | again loop
text: 000A07B0 | cmp %i5, %i1
text: 000A07B4 |
Loading IDP module C:\IDA\sparc.w32 for processor sparc...OK
Loading type library...
Autanalysis subsystem is initialized.
Database for file 'libns-httpd40.so' is loaded.
Compiling file 'C:\IDA\idc\ida.idc'...
Executing function 'main'...
Search completed
All: idle Disk: 2GB 000A073C 000A073C: __0fJCHttpUtilCanonicalizeURIPathPcRt

```

Diving a bit deeper into the program, we find another function that is used to check the format "cleaned" request. As if the idea of looking for bad input isn't ridiculous enough on its own, this function is actually named `INTutil_uri_is_evil_internal` (what fun!). This additional function supposed to trap malicious hackers who are attacking the system. The call should return TRUE or FALSE depending on whether the URI is determined to be "evil." This is greatly amusing, so let's reverse engineer this call. Obviously, we must be able to get past this call during any real attack. The IDA reverse of the function looks something like this:

```
.text:00056140 ! ||||| S U B R O U T I N E
```

```

.text:00056140
.text:00056140
.text:00056140          .global INTutil_uri_is_evil_internal
.text:00056140 INTutil_uri_is_evil_internal:
.text:00056140          ldsb    [%o0], %o1
.text:00056144          mov     1, %o3
.text:00056148          mov     2, %o4
.text:0005614C          cmp     %o1, 0
.text:00056150          be, pn   %icc, loc_561F4
.text:00056154          mov     %o0, %o5

...

.text:00056158          mov     %o2, %o0
.text:0005615C          mov     0, %o2
.text:00056160          cmp     %o1, 0x2F
.text:00056164
.text:00056164 loc_56164:
.text:00056164          bne, a   %icc, loc_561DC
...

```

We set a break point and examine the data going into this call as follows:

```

(gdb) x/8s $o0
0x97f030:      "/usr/local/iplanet/servers/docs/test_string.greg//"

```

```
0x97f064:      "ervers/docs"
0x97f070:      "/usr/local/iplanet/servers/docs"
0x97f090:      " "
0x97f091:      "\2272\230"
0x97f095:      " "
0x97f096:      " "
0x97f097:      " "
```

In this example, our break point fires after we supplied the following URL:

```
http://172.16.10.10/test_string.greg/%2F//.
```

At this point we can see that the hex-encoded characters in the URI have already been converted the time it has reached this point. Through some further probing, we also note that the "evil" check never made for the following URL:

```
http://172.16.10.10/../../../../../../../../etc/passwd
```

That is, when we directly access the password file, some check occurs in the program that denies the request before the "evil" check even runs. We never make it to the "evil" check! Clearly, there are multiple points in the program that are checking our input for hostility.

Interestingly, when the path is prefixed with a subdirectory, we do land in the "evil" check:

`http://172.16.10.10/sassy/../../../../../../../../etc/passwd`

The subdirectory "sassy" in this case is not required to exist. The critical insight is that we are confusing the logic of the program. By placing a bogus subdirectory in the path, the logic branches differently than if a direct request is made for the password file.

This means we have defeated the first check on our input. When multiple checks and branches are to be occurring like this, this is a good indication that you will eventually find a way into the program. A better designed program will usually have a single cohesive point where a check or set of checks occurs. (Note that in a few interesting cases, no checks are needed because the target program is CHROOTed or uses some other security mechanism.)

Misclassification

Classification is very important in software. Once a classification decision is made, a whole set of logic executes. Thus, mistakes in classification can be deadly.

Software relies very heavily on classification. Once a root decision is made, software makes calls to particular modules and/or runs through large sets of subroutines. A good example of request classification and its inherent dangers involves the way HTTP servers decide what kind of file is being requested: Scripts are to be handled by the scripting engine, executables by the cgi engine, and regular text files by the regular text file engine. Malicious hackers figured out a long time ago how to request a file while fooling the Web server into believing the file was something else entirely. The most pervasive use of this technique involves stealing binaries of cgi programs, or script files that contain hard-coded passwords and other interesting logic.

Attack Pattern: Cause Web Server Misclassification

A very famous set of classification problems occurs when a Web server examines the last few characters of a filename to determine what kind of file it is. There are many ways to take advantage of these kinds of problems—appending certain strings to file names, adding dots, and so forth.

* Attack Example: Misclassification in NTFS File Streams Specifier

One Web server misclassification bug is exercised by appending the string `::$DATA` to the end of a filename. The Web server code looks at the last three characters in the string and sees `ATA`. As a result, if you request `/index.asp::$data`, the Web server fails to detect that what is being requested is an ASP file, and happily returns the contents of the file (revealing some logic best left hidden from attackers). The "asp dot" bug is another example of misclassification.

Building "Equivalent" Requests

A large number of commands are subject to parsing or filtering. In many cases a filter only considers a particular way to format a command. The fact is that the same command can usually be encoded thousands of different ways. In many cases, an alternative encoding for the command will produce the same results as the original command. Thus, two commands that look different from the log perspective of a filter end up producing the same semantic result. In many cases, an alternative command can be used to attack a software system, because the alternative command allows an application to perform an operation that would otherwise be blocked.

Mapping the API Layer

A good approach to help identify and map possible alternate encodings involves writing a small program that loops through all possible inputs to a given API call. This program can, for example, attempt to open filenames in a variety of ways. For each iteration of the loop, the "mungified" filename can be passed to the API call and the result noted.

The following code snippet loops through many possible values that can be used as a prefix to the filename `test.txt`. Results of running a program like this can help us to determine which characters can be used to perform a `../..` (dots and slashes) relative traversal attack.

```
int main(int argc, char* argv[])
{
    for(unsigned long c=0x01010101;c != -1;c++)
    {
        char _filepath[255];

        sprintf(_filepath, "%c%c%c%c\\test.txt", c >> 24, c >> 16, c >> 8, c&0xf);

        try
        {
            FILE *in_file = fopen(_filepath, "r");

            if(in_file)
            {
```

```

        printf("checking path %s\n", _filepath);

        puts("file opened!");

        getchar();

        fclose(in_file);

    }

}

catch(...)

{

}

}

return 0;

}

```

Slight (but still automatic) modifications can be made to the string in creative ways. Ultimately, modified string boils down to an attempt to use different tricks to obtain the same file. For example, the resulting attempt might try a command like this:

```

sprintf(_filepath, "..%c\\..%c\\..%c\\..%c\\scans2.txt", c, c, c, c);

```

A good way to think about this problem is to think of layers. The API call layer is what the exam here are mapping. If an engineer has placed any filters in front of the API call, then these filters are considered additional layers, wrapping the original set of possibilities. By pondering all the possibilities that can be provided at the API layer, we can begin uncovering and exercising any filters that the software has in place. If we know that the software definitely uses file API calls, we can try all kinds of file encoding tricks that we know about. If we get lucky, eventually one set of encoding tricks will work and we can get our data successfully through the filters and into the API call.

Drawing on the techniques described in [Chapter 5](#), we can list a number of possible escape codes that can be injected into API calls (many of which help with the filter avoidance problem). If the data are being piped into a shell, for example, we might be able to get control codes to take effect. A pair of characters may write data to a file or a stream that are eventually meant to be viewed on a terminal or in

program. As a simple example, the following string contains two backspace characters that are show up in the terminal's execution:

```
write("echo hey!\x08\x08");
```

When the terminal interprets the data we have passed in, the output will be missing the last two of the original string. This kind of trick has been used for ages to corrupt data in log files. Log files contain all kinds of data about a transaction. It may be possible to insert NULL characters (for example, `'\0'`) or to add so many extra characters to the string that the request is truncated in the log. Imagine a request that has more than a thousand extra characters tacked on at the end. Ultimately, the string is trimmed in the log file, and the important telltale data that expose an attack will be lost.

Ghost Characters

Ghost characters are extra characters that can be added to a request. The extra characters are there to affect the validity of the request. One easy example involves adding extra slashes to a filename. In some cases, the strings

```
/some/directory/test.txt
```

and

```
//////////some//////////directory//////////test.txt
```

are equivalent requests.

Attack Pattern: Alternate Encoding the Leading Ghost Characters

Some APIs will strip certain leading characters from a string of parameters. Perhaps these characters are considered redundant, and for this reason they are removed. Another possibility is the parser logic at the beginning of analysis is specialized in some way that causes some characters to be removed. The attacker can specify multiple types of alternative encodings at the beginning of a string as a set of probes.

One commonly used possibility involves adding ghost characters—extra characters that don't affect the validity of the request at the API layer. If the attacker has access to the API libraries being targeted, certain attack ideas can be tested directly in advance. Once alternative ghost encodings emerge through testing, the attacker can move from lab-based API testing to testing real-world service implementations.

* Attack Example: Alternate Encoding with Ghost Characters in FTP and Web Server

A good example covering the use of alternate encodings and ghost characters can be found in Web server implementations. A number of implementations filter for attempts to carry out a directory traversal attack. In some exploits, if the attacker supplies a string such as `.../.../.../winnt`, it will fail to filter things properly and the attacker will illegally gain access to a "protected" directory. This kind of attack lies in supplying the leading `"..."` (note the three dots). This is commonly referred to as *triple-dot vulnerability*, even though it is indicative of a problem far more serious than consuming dots.

Using the file system API as the target, the following strings are all equivalent to many programs

```
.../.../.../test.txt
...../.../.../test.txt
..?/.../.../test.txt
..???????/.../.../test.txt
../test.txt
```

As you can see, there are many ways to make a semantically equivalent request. All these strings result in a request for the file `../test.txt`.

* Attack Example: Alternate Encoding Triple-dot in SpoonFTP

By using the triple dot, the attacker can traverse directories on SpoonFTP V1.1:

```
ftp> cd ...  
  
250 CWD command successful.  
  
ftp> pwd  
  
257 "/..." is current directory.
```

Equivalent Meta-characters

Delimiting characters are also special. They are used to separate commands or words in a request. Servers tend to look for delimiters to determine how a command chunks up. When attacking a target AF, a commonly used technique involves adding extra commands and causing them to execute. For this, understanding how to encode delimiting characters is of particular interest. A filter may be removed by otherwise watching for certain delimiting characters. Spotting a command separator in untrusted input is usually a dead giveaway that someone is attempting to insert extra commands.

Consider the space character used to separate words (as in this sentence). Many software systems accept the tab character as a replacement for the space. To the program, white space is white space.

Attack Pattern: Using Slashes in Alternate Encoding

Slash characters provide a particularly interesting case. Directory-driven systems, such as file systems and databases, typically use the slash character to indicate traversal between directories or other container components. For murky historical reasons, PCs (and, as a result, Microsoft OSs) choose to use a backslash, whereas the UNIX world typically makes use of the forward slash. The schizophrenic result is that many MS-based systems are required to understand both forms of the slash. This gives the attacker many opportunities to discover and abuse a number of common filtering problems. The goal of this pattern is to discover server software that only applies filters to one version, but not the other.

* Attack Example: Slashes in Alternate Encodings

The two following requests are equivalent on most Web servers:

```
http://target server/some_directory\..\..\..\winnt
```

is equivalent to

```
http://target server/some_directory/../../../../winnt
```

Multiple encoding conversion problems can also be leveraged as various slashes are instantiated encoded, UTF-8, or unicode. Consider the strings

```
http://target server/some_directory\..\%5C..\%5C..\winnt
```

where `%5C` is equivalent to the `\` character.

Escaped Meta-characters

Many filters look for all meta-characters, but may miss some if they are "escaped." An escape character usually precedes a special sequence of characters. The special sequence will either be converted to a control character or it will be treated as a control character later in the input stream.

Here is an example of how escape characters might be filtered. Note that testing is required to confirm actual behavior:

`ESCn` where `ESC` is left in place and `n` is left in place as a normal character

`ESCn` where `ESC` is stripped and `n` is left in place as a normal character

(Substitute `n` with a carriage return or a null byte.)

Attack Pattern: Using Escaped Slashes in Alternate Encoding

Providing a backslash as a leading character often causes a parser to believe that the *next* character is special. This is called *an escape*. For example, the byte pair `\0` might result in a single zero byte (a NULL) being sent. Another example is `\t`, which is sometimes converted into a tab character. There is often an equivalent encoding between the back slash and the escaped back slash. This means that `\\` results in a single forward slash. A single forward slash also results in a single forward slash. The encoding table looks like this:

/	/
\\	/

Having two alternate ways to encode the same character leads to filter problems and opens avenues to attack.

* Attack Example: Escaped Slashes in Alternate Encodings

An attack leveraging this pattern is very simple. If you believe the target may be filtering the slash to supply `\\` and see what happens. Example command strings to try out include

```
CWD ..\\..\\..\\..\\winnt
```

which converts in many cases to

```
CWD ../../../../winnt
```

To probe for this kind of problem, a small C program that uses string output routines can be very useful. File system calls make excellent testing fodder. The simple snippet


```
int main(int argc, char* argv[])
{
    puts("\\ / \\ \\ \\ ? \\ . \\ | ");
    return 0;
}
```

produces the output

```
/ \ ? . |
```

Clearly, the back slash is ignored, and thus we have hit on a number of alternative encodings to with. Given our previous example, we can extend the attack to include other possibilities:

In this case, the attacker is attempting to traverse to a directory that is not supposed to be part of Web services. The trick is fairly obvious, so many Web servers and scripts prevent it. However, alternate encoding tricks, an attacker may be able to get around badly implemented request filters.

In October 2000, a hacker publicly revealed that Microsoft's IIS server suffered from a variation problem. In the case of IIS, all the attacker had to do was provide alternate encodings for the slashes found in a classic attack. The unicode translations are

.	C0 AE
/	C0 AF
\	C1 9C

Using this conversion, the previously displayed URL can be encoded as

http://target.server/some_directory/%C0AE/%C0AE/%C0AE%C0AE/%C0AE%C0AE/winnt

Attack Pattern: UTF-8 Encoding

UTF-8 is a system for encoding characters using a variable number of bytes. Instead of simply using 2 bytes as in unicode, a character can be encoded with 1, 2, or even 3 bytes. The characters described in the previous unicode subgroup are shown here encoded using three bytes in UTF-8:

.	F0 80 AE
/	E0 80 AF
\	F0 81 9C

The RFC that defines UTF-8 encoding is RFC-2044. UTF-8 makes a decent target for the same reasons that unicode does.

Attack Pattern: URL Encoding

In many cases, a character can be encoded as `%HEX-CODE` in URL strings. This has led to a number of classic filtering problems.

* Attack Example: URL Encodings in IceCast MP3 Server

The following type of encoded string has been known to traverse directories against the IceCast M

[9] For more information, go to <http://www.securitytracker.com/alerts/2001/Dec/1002904.html>.

`http://[targethost]:8000/somefile/%2E%2E/target.mp3`

or using `"/%25%25/"` instead of `"../"`.

* Attack Example: URL Encodings in Titan Application Firewall

The Titan application firewall fails to decode hex-encoded and URL-encoded characters. For example, it does not filter `%2E`.

Many other examples of alternate encoding exist. These include UCS-2 unicode, HTML escape codes, and even such trivial changes involving character case problems and converting spaces to tab characters.

All these encoding situations lead to possible encoding fun.

Attack Pattern: Alternative IP Addresses

IP address ranges can be represented using alternative methods. Here are some examples:

`192.160.0.0/24`

`192.168.0.0/255.255.255.0`

`192.168.0.*`

Classic encoding attacks can be directed against IP numbers as well.

* Attack Example: Dotless IP Addresses in Internet Explorer

Alternate encoding of IP numbers poses problems to filters and other security measures that do not interpret values properly such as ports and IP addresses. URL filtering in general is plagued with problems. The Microsoft Internet Explorer package allows specification of the IP address in a variety of number formats.^[10] Here are some equivalent ways to request the same Internet Web site:

[10] For more on this issue, go to <http://www.securitytracker.com/alerts/2001/Oct/1002531.html>.

<http://msdn.microsoft.com>

<http://207.46.239.122>

<http://3475959674>

Combined Attacks

Ultimately, all of the tricks described here can be combined in various ways.

Attack Pattern: Slashes and URL Encoding Combined

Combine two (or more) encoding tricks.

* Attack Example: Combined Encodings CesarFTP

Alexandre Cesari released a freeware FTP server for Windows that fails to provide proper filtering of multiple encoding. The FTP server, CesarFTP, included a Web server component that could be attacked with a combination of the triple-dot and URL encoding attacks.

An attacker could provide a URL that included a string like

`/...%5C/`

This is an interesting exploit because it involves an aggregation of several tricks—the escape character encoding, and the triple dot.

Audit Poisoning

Up to this point we have focused on attacks against filters or the classification decisions made by servers. Another area in which character obfuscation comes in handy is in manipulating log files. There are many real-world examples in which attackers confuse the log file to escape detection. This is an excellent technique to avoid creating trusted forensic evidence.

Attack Pattern: Web Logs

Escape characters are often translated before being printed to the log file. For example, under IIS the string `/index%2Easp` is recorded in the log file as `/index.asp`. A more complicated string can be used to record fake entries in the logfile. For example:

```
/index.asp%FF200%FFHTTP/1.1%0A00:51:11%FF[192.168.10.10]%FFGET%FF  
%FFcgi-bin/phf
```

This string will force a carriage return in the log file and forge a fake entry showing the address `192.168.10.10` getting the `cgi-bin/phf` file.

This kind of problem has taken many forms over the years. In the worst cases, exploits have been written that will execute when the log file is piped through `grep` or some other log file analysis script. In this case, the attack is aimed squarely at a security mechanism. Clearly, many layers of encoding and interpreting can get involved here. For those organizations using simple log file analysis, here is a simple question: Can you trust the characters in your log file?

Note that only log analysis tools that "do stuff" with active content will be susceptible to attacks like this. Simple tools like `grep` are unlikely to suffer from such problems. Of course, even simple tools may have bugs or flaws that can be exploited (the fun part being that such tools are often invoked from root or administrator).

Conclusion

At the beginning of this chapter, we invoked the complexity problem in open dynamical systems, and then went on to discuss the complicated ways that input can influence state in computer software. Throughout the chapter we provided supporting evidence for our complexity claims through specific examples, showing how input can be specially crafted so that it can subvert filtering mechanisms and run-of-the-mill IDS equipment. But we've really only scratched the surface.

Security problems related to state over time (the dynamics of a system) are destined to become more and more difficult and relevant as well-worn, easy-to-identify bugs such as buffer overflows are eradicated in code. All good attackers know to examine state very closely and to measure how user input can be used to induce insecure states. As systems become more distributed, attacks will with more regularity take advantage of race conditions and state desynchronization between far-flung parts. Solving these difficult problems will require another generation of tools, more sophisticated techniques, and some creative imagination.